

Cybersecurity Packet Sniffer

Description

This project provides a simple yet effective Python-based packet sniffer designed for cybersecurity analysis and network monitoring. Utilizing the Scapy library, it captures and dissects network traffic, offering insights into various protocols such as Ethernet, IP, TCP, UDP, ICMP, HTTP, and DNS. The tool supports filtering traffic using Berkeley Packet Filter (BPF) syntax and can log captured packet information to a file, making it suitable for educational purposes, network diagnostics, and basic security auditing.

Features

- **Packet Capture:** Sniffs live network traffic on a specified interface.
- **Protocol Dissection:** Parses and displays information for:
 - Ethernet (MAC addresses)
 - IP (Source/Destination IP, Protocol)
 - TCP (Source/Destination Ports, Flags)
 - UDP (Source/Destination Ports)
 - ICMP (Type)
 - HTTP (Requests and Responses)
 - DNS (Queries and Responses)
- **Traffic Filtering:** Supports BPF (Berkeley Packet Filter) syntax for targeted packet capture (e.g., `tcp port 80` , `host 192.168.1.1`).
- **Logging:** Option to save captured packet details to a specified log file.
- **Command-Line Interface (CLI):** Easy-to-use command-line arguments for configuration.

Installation

Prerequisites

- Python 3.x
- `pip` (Python package installer)
- Root privileges are required to run the sniffer.

Steps

1. Clone the repository (or create the project directory and files):

Bash

```
mkdir ~/packet_sniffer_project
cd ~/packet_sniffer_sniffer_project
# Copy sniffer.py into this directory
```

2. **Install Scapy:** The primary dependency for packet manipulation.

Bash

```
sudo pip3 install scapy
```

Usage

To run the packet sniffer, navigate to the project directory and execute the `sniffer.py` script with `sudo`.

Bash

```
sudo python3 sniffer.py [OPTIONS]
```

Options

- `-i` , `--interface` : Specify the network interface to sniff on (e.g., `eth0` , `wlan0`). If not specified, Scapy attempts to sniff on the default interface.
- `-c` , `--count` : Number of packets to capture. Set to `0` (default) for infinite capture until manually stopped (Ctrl+C).
- `-f` , `--filter` : Apply a BPF filter expression to capture specific traffic (e.g., `'tcp port 80'` , `'udp and port 53'`).
- `-l` , `--log` : Specify a path to a log file where captured packet information will be saved.

Examples

1. **Basic Sniffing on `eth0` (infinite capture):**

Bash

```
sudo python3 sniffer.py -i eth0
```

2. Capture 10 packets on wlan0 :

Bash

```
sudo python3 sniffer.py -i wlan0 -c 10
```

3. Sniff HTTP traffic (port 80) on eth0 and log to a file:

Bash

```
sudo python3 sniffer.py -i eth0 -f 'tcp port 80' -l logs/http_traffic.log
```

4. Monitor DNS traffic (port 53 UDP) on any interface:

Bash

```
sudo python3 sniffer.py -f 'udp port 53'
```

Example Output

Plain Text

```
[*] Starting packet sniffer on interface: eth0
[2026-06-25 07:38:51] Ether da:76:99:5b:50:e5 -> 02:fc:00:00:00:05 | IP 142.250
[2026-06-25 07:38:51] Ether da:76:99:5b:50:e5 -> 02:fc:00:00:00:05 | IP 142.251
[2026-06-25 07:38:51] Ether 02:fc:00:00:00:05 -> da:76:99:5b:50:e5 | IP 169.254
[2026-06-25 07:38:51] Ether 02:fc:00:00:00:05 -> da:76:99:5b:50:e5 | IP 169.254
[2026-06-25 07:38:51] Ether 02:fc:00:00:00:05 -> da:76:99:5b:50:e5 | IP 169.254
```

License

This project is open-source and available under the MIT License. Feel free to modify and distribute.